EcoWatch SJDM

VLM Verification Layer

Implementation plan: two-layer AI verification (VLM gate + Mask R-CNN) para sa citizen reports

Draft v1 | October 2026 | Next.js + FastAPI + Supabase + Hugging Face Spaces

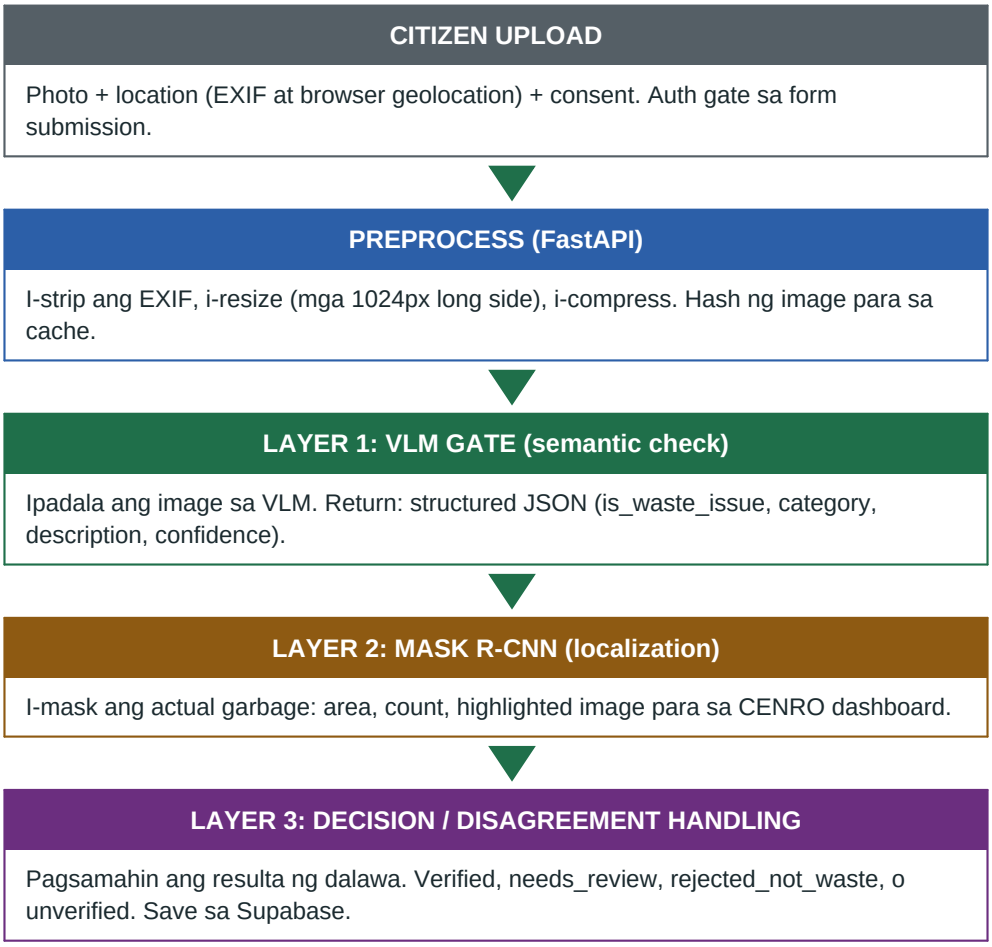
1. Ang Problema

Ang Mask R-CNN ay isang **instance segmentation model**. Natututo lang siya ng shapes at textures ng mga class na na-train sa kanya, kaya wala siyang tunay na *understanding* ng context. Kapag ang litrato ay pagkain sa loob ng plastic (halimbawa, baon na nasa plastic container), ang nakikita lang niya ay yung plastic, kaya madalas na lumalabas na **garbage**. Ito ay false positive na hindi ma-aayos ng pag-tune lang ng threshold.

Ang Mask R-CNN ay magaling sa tanong na **"nasaan at anong hugis ng basura?"** pero mahina sa tanong na **"basura nga ba talaga ito at bakit?"**. Ang pangalawang tanong ay trabaho ng isang Vision-Language Model (VLM), gaya ng Gemini, Claude, o GPT, na kayang tumingin sa litrato at magpaliwanag kung ano ang laman nito.

> **Solusyon sa isang pangungusap**
> Maglagay ng VLM bilang **first layer (semantic gate)** na magdedesisyon kung basura nga ang litrato, tapos Mask R-CNN bilang **second layer** para i-mask at i-localize lang ang basura. Kapag nag-disagree ang dalawa, mapupunta sa CENRO para sa manual review.

2. Ang Pipeline



Layer 1: VLM gate (semantic check)

Magpapadala ka ng image at magpapa-return ka ng structured JSON:

```
{
  "is_waste_issue": true,
  "category": "plastic | food_waste | mixed | hazardous | not_waste",
  "description": "pile of plastic bottles beside a canal",
  "confidence": "high | medium | low"
}
```

Kapag **not_waste** (na may mataas na confidence), i-reject agad o i-flag. Hindi na dadaan sa Mask R-CNN, kaya nakakatipid ka rin sa compute.

Layer 2: Mask R-CNN (localization)

Dito lang siya gagamitin para i-mask yung actual garbage. Gagamitin ang output para sa area, count, at highlighted na image sa CENRO dashboard. Hindi na siya ang magdedesisyon kung *valid* ang report. Trabaho na iyon ng Layer 1 at Layer 3.

Layer 3: Disagreement handling

Kapag nag-disagree yung dalawa (VLM: not waste, Mask R-CNN: garbage detected), **huwag auto-reject**. I-send sa CENRO para sa manual review. Mas ligtas ito kaysa magkamali ang AI sa civic report.

Paglilinaw sa flow (importante)
Para magkaroon ng "disagreement", kailangang tumakbo pa rin ang Mask R-CNN sa mga kasong hindi sigurado ang VLM. Kaya ang rule: **skip lang ang Mask R-CNN kapag VLM = not_waste na may HIGH confidence**. Kapag medium o low ang confidence ng not_waste, patakbuhin pa rin ang Mask R-CNN para makita kung mag-disagree sila.

Decision table

VLM result	Mask R-CNN	Status	Ano ang mangyayari
Waste (high/medium)	May na-detect	**verified**	Tuloy sa CENRO dashboard na may mask at description.
Waste (high/medium)	Walang na-detect	**needs_review**	Posibleng maliit o malabo ang basura. Manual review.
not_waste (HIGH)	Hindi pinatakbo	**rejected_not_waste**	Ipaliwanag sa citizen kung bakit. Payagan ang resubmit.
not_waste (medium/low)	May na-detect	**needs_review**	Disagreement. Manual review ng CENRO.
not_waste (medium/low)	Walang na-detect	**rejected_not_waste**	Parehong AI ang nagsasabing hindi basura.
VLM unavailable / failed	Kahit ano	**unverified**	Tanggapin pa rin ang report. Review queue, mas mababang priority.

3. Aling AI ang Gagamitin?

Option	Lakas	Kahinaan / Note
Gemini Flash / Flash-Lite	Mura, mabilis, magaling sa image description at structured JSON. Kadalasan may free tier na swak sa student project.	I-check ang current limits at pricing sa official docs, nagbabago ito.
Claude (Haiku / Sonnet)	Magaling sa vision at sa pagsunod sa strict JSON format.	Mas mahal kumpara sa Flash.
GPT-4o-mini or similar	Okay din sa vision, parehong approach.	Paid. Gagamitin lang bilang alternatibong provider.
CLIP / SigLIP (open-source, zero-shot)	Pwedeng i-run sa HF Space mismo. Walang API cost at walang privacy issue sa third party.	Hindi nagde-describe, nagre-rank lang ng labels. Mas mahina sa tricky cases. Tingnan ang RAM ng Space kasama ng TensorFlow.

Rekomendasyon: Gemini Flash ang unang subukan, Claude Haiku o OpenAI mini bilang secondary, at CLIP bilang huling local fallback. Pero hindi dapat opinyon ang batayan. Gumawa ng eval set (Phase 0) at i-compare ang mga provider sa sarili mong data: accuracy at cost per image.

4. Step-by-Step Build

Phase 0: Eval set muna

- Mag-ipon ng 100 hanggang 200 images: malinaw na basura, food sa plastic, malinis na kalsada, blurry, gabi, may tao, at iba pang tricky cases.
- Lagyan ng tamang label (expected category at is_waste_issue). Ito ang gagamitin mo para mag-test at mag-tune ng prompt.
- Gumawa ng simpleng CSV: filename, expected_label, notes.

Done kapag: *may folder ng labeled images na may variety ng tricky cases.*

Phase 1: Kumuha ng API key

- Gemini: Google AI Studio, mag-generate ng key.
- Ilagay sa **Hugging Face Space Secrets**. Hindi sa code, hindi sa repo, hindi sa Vercel frontend.
- Para sa local: ilagay sa `.env` at i-add sa `.gitignore`.
- Optional pero recommended: gumawa rin ng key sa pangalawang provider (Claude o OpenAI) para sa fallback.

Done kapag: *gumagana ang isang simpleng test call at wala ang key sa git history.*

Phase 2: I-define ang contract

- Gumawa ng Pydantic model para sa response, para ma-validate ang output ng VLM.
- Isulat ang prompt: malinaw na definition ng "waste issue", mga edge case (halimbawa, "food sa plastic container na nasa kamay o mesa = not waste; kung nakakalat o tapon sa kalsada = waste"), at i-require ang JSON-only output.
- Gumamit ng structured output / JSON mode ng provider kung meron.

```
from pydantic import BaseModel
from typing import Literal

class VLMResult(BaseModel):
    is_waste_issue: bool
    category: Literal["plastic", "food_waste", "mixed", "hazardous", "not_waste"]
    description: str
    confidence: Literal["high", "medium", "low"]
```

Done kapag: *may schema at prompt na nagbibigay ng valid JSON sa 10 sample images.*

Phase 3: Gumawa ng vlm_service.py sa FastAPI

Isang module lang na may isang function: `verify_image(image_bytes) -> VLMResult`.

- **Preprocess:** i-strip ang EXIF, i-resize (mga 1024px sa long side), i-compress. Mas mura at mas mabilis, at privacy-safe pa.
- Tawagin ang API na may **timeout** (mga 10 hanggang 15 segundo).
- I-validate ang response gamit ang Pydantic. Kapag sira ang JSON, treat as failure (hindi crash).
- Gumawa ng **provider interface** para swap lang ang provider mamaya (kailangan ito sa fallback chain).

Done kapag: *kaya mong tawagin ang* `verify_image()` *mula sa isang test script at laging valid ang return o malinaw ang error.*

Phase 4: I-integrate sa report flow

```
upload -> preprocess -> Layer 1 (VLM) -> Layer 2 (Mask R-CNN) -> decision -> save
```

Status	Kahulugan	Sino ang hahawak
verified	Pareho ang resulta ng VLM at Mask R-CNN	CENRO (normal queue)
needs_review	Disagreement o hindi sigurado	CENRO (review queue)
rejected_not_waste	Hindi basura ayon sa AI	Citizen (puwedeng mag-resubmit)
unverified	Walang VLM (down o naubos ang quota)	CENRO (mas mababang priority)

- I-save sa DB ang `vlm_result` (JSON), `vlm_provider`, at timestamp para ma-audit.
- **Huwag i-block ang citizen** habang naghihintay ng AI. Tanggapin at i-save muna ang report, tapos mag-verify sa background kung kailangan.

Done kapag: *end-to-end na tumatakbo ang isang report sa lahat ng status path.*

Phase 5: Admin dashboard (CENRO)

- Review queue para sa `needs_review` at `unverified`.
- Ipakita ang VLM description at ang Mask R-CNN mask **side by side**, para mabilis mag-decide ang CENRO.
- Approve / reject buttons na may reason, at i-log kung sino ang nag-decide.

Done kapag: *kaya ng CENRO user na mag-review ng isang disagreement case sa ilang click lang.*

Phase 6: Test at i-tune

- Patakbuhin ang eval set sa script (command line lang, walang website na kailangan).
- Tingnan ang mga mali, i-adjust ang prompt, ulitin.
- Dito mo makikita kung kaya na ng Flash-Lite o kailangan ng mas malakas na model.
- I-compare ang mga provider: accuracy, latency, at cost per image.

Done kapag: *alam mo ang accuracy ng bawat provider sa eval set at napili mo na ang primary.*

Phase 7: Monitoring

- Mag-log ng bilang ng calls per day, error rate, at kung ilang beses nag-fallback.
- Mag-set ng simpleng alert (log o email) kapag may lampas na sa threshold ang fallback o unverified count.
- Kung wala kang visibility, hindi mo malalaman na naubos na pala ang quota.

Done kapag: *may paraan kang makita ang usage at fallback rate nang hindi nagbabasa ng raw logs.*

5. Fallback Strategy

Isipin ang fallback bilang hagdan. Kapag bumagsak ang isang hakbang, bababa ka sa susunod. Layunin: **hindi mawawala ang report** kahit pumalya ang AI.

#	Hakbang	Detalye
1	**Protektahan ang quota mo**	Cache by image hash (pareho ang image, hindi na tatawag ulit). Rate limit per user/IP sa backend para hindi ma-spam. Daily budget counter: kapag malapit na sa limit, lumipat na sa fallback bago pa mag-error.
2	**Retry na may backoff**	Para sa 429 (rate limit) at 5xx. 2 hanggang 3 tries lang, hindi habambuhay.
3	**Ibang model sa parehong provider**	Halimbawa Flash papuntang Flash-Lite. Madalas magkahiwalay ang quota per model, pero i-check sa docs.
4	**Ibang provider**	Claude Haiku o OpenAI mini gamit ang parehong prompt at parehong Pydantic schema.
5	**Local CLIP / SigLIP zero-shot**	Walang limit at walang API cost. Mas mahina, kaya gamitin bilang rough check at mas mababang confidence ang ibigay.
6	**Mask R-CNN lang**	Huling hagdan. I-mark ang report bilang `unverified`, papasok sa CENRO queue na mas mababa ang priority.

Skeleton ng fallback chain

```
PROVIDERS = [gemini_flash, gemini_flash_lite, claude_haiku, local_clip]

async def verify(image):
    for provider in PROVIDERS:
        try:
            return await provider.verify(image)   # may timeout + validation
        except (RateLimit, Timeout, InvalidResponse):
            continue
    return None   # -> status = "unverified"
```

Dalawang rules na hindi pwedeng labagin

1. Hindi dapat mawala ang report dahil down ang AI. Tanggapin, i-save, at mag-verify sa background kung kailangan. Huwag i-block ang citizen.
2. Kapag nagde-degrade na (nasa fallback na), dapat may alert sa iyo. Kahit simpleng log o email lang, para hindi mo malaman after two weeks na puro `unverified` na pala.

6. Local Testing vs Production

Local muna dapat, hindi sa published website. Mas mabilis mag-debug at hindi mo mauubos ang quota sa production.

Paano mag-test locally

- **Backend:** `uvicorn main:app --reload`, tapos buksan ang `http://localhost:8000/docs`. May built-in Swagger UI kung saan puwede kang mag-upload ng image at makita ang response nang walang frontend.
- **Frontend:** `npm run dev`, tapos ituro sa local backend gamit ang `.env.local` (`NEXT_PUBLIC_API_URL=http://localhost:8000`). I-allow ang `localhost:3000` sa CORS ng FastAPI.
- **Database:** SQLite lang muna sa local.

- **Eval script:** daanan ang lahat ng images sa folder, tawagin ang `verify_image()`, mag-output ng CSV (filename, expected, predicted, confidence).
- **Fallbacks:** mas madali pang i-test locally. Sadyang sirain ang primary key (simulated failure), maglagay ng sleep para sa timeout, o mag-raise ng fake 429 para makita kung bumababa nga sa susunod na hagdan.
- **Phone test:** buksan ang local server sa same Wi-Fi gamit ang IP ng laptop, o gumamit ng tunnel tool. Huwag i-expose nang matagal ang dev server.

Mga bagay na mag-iiba sa production

Bagay	Bakit kailangang i-test sa live
Resources	Mas malakas ang laptop kaysa sa free HF Space. Posibleng mag-OOM o bumagal doon, lalo na kung TensorFlow, Mask R-CNN, at CLIP ay magkakasama sa isang container.
Cold starts	Natutulog ang HF Spaces kapag walang gumagamit, kaya mas mabagal ang unang request.
Env variables at CORS	Dito madalas pumapalpak pagka-deploy.
Real latency	Iba ang bilis ng network mula Spaces papunta sa API kumpara sa local.

Flow: local test, deploy, mabilis na smoke test sa live (1 hanggang 2 images lang), saka i-announce.

7. Mga Babala (Honest Notes)

1. Privacy at DPA (RA 10173)
Ipapadala mo ang photos ng citizens sa third-party API. I-strip ang EXIF (GPS at metadata) bago ipadala, para hindi lumabas sa Layer 1 ang location data. Ilagay sa privacy notice at consent flow na may third-party AI processing. Isipin din na ang litrato ay maaaring may mukha ng tao o plaka ng sasakyan. Kung kaya, mag-blur o i-minimize ang data na ipinapadala. Para sa final compliance, kumonsulta sa taong may alam sa DPA o sa DPO ng LGU.

2. API key security
Sa FastAPI backend lang dapat ang API call, hindi sa Next.js frontend, para hindi ma-leak ang key. Gumamit ng secrets (HF Space Secrets) at huwag i-commit ang `.env`.

3. Nagkakamali rin ang VLM
Nagha-hallucinate at inconsistent minsan ang VLM. Kaya may confidence field, validation ng schema, at manual review path. Iturin ang output ng VLM bilang **untrusted data**: i-validate ito at huwag gamitin nang direkta sa query o sa anumang bagay na sensitibo. May mga larawan ding may nakasulat na teksto na puwedeng sumubok mag-manipula sa model.

4. Fallback at quota
Isipin kung ano ang mangyayari kapag down o rate-limited ang API. Dapat may queue o fallback sa Mask R-CNN-only na may "unverified" flag. Huwag ipagpalagay na pareho ang quota ng lahat ng model.

5. Abuse at spam
Dahil may quota at cost ang bawat call, puwedeng abusuhin ang endpoint. Auth gate sa submission (gaya ng nasa architecture mo na), rate limit per user, at image size limit.

6. Root cause: training data ng Mask R-CNN

Puwede ring i-improve ang Mask R-CNN mismo sa pamamagitan ng pag-add ng **hard negatives** (food sa plastic, etc.) sa training data. Hindi ito kapalit ng VLM layer, pero nakakabawas ng false positives sa mismong model. Kung may nagre-review na CENRO, ang mga `needs_review` cases ay puwedeng maging bagong training data.

7. Cost at limits

Ang mga detalye ng free tier, pricing, at model names ay nagbabago. I-verify palagi sa official docs ng provider bago mag-commit, at huwag i-hardcode ang pagpapalagay na libre ito habambuhay.

8. Master Checklist

	Task
[ ]	Phase 0: eval set (100 hanggang 200 labeled images) tapos na
[ ]	Phase 1: API key (primary at secondary) nasa Secrets, wala sa git
[ ]	Phase 2: Pydantic schema at prompt na may edge cases
[ ]	Phase 3: vlm_service.py na may timeout, validation, at provider interface
[ ]	Phase 4: integrated sa report flow, may apat na status, naka-save ang vlm_result
[ ]	Phase 5: CENRO review queue na may side-by-side na VLM at mask
[ ]	Phase 6: eval run, na-tune ang prompt, napili ang primary provider
[ ]	Phase 7: logging at alert para sa quota at fallback rate
[ ]	Fallback chain: cache, retry, secondary model, secondary provider, CLIP, Mask R-CNN-only
[ ]	Privacy: EXIF stripped, consent at privacy notice updated para sa third-party AI
[ ]	Local test tapos na, tapos smoke test sa live deployment

Dokumentong ito ay planning guide mula sa usapan natin. Hindi ito legal advice. Para sa DPA compliance, kumonsulta sa kwalipikadong tao.